

while文

条件が正しい間だけループを行う

```
while 条件  
  処理  
end
```

何回ループしても処理が終わらない「無限ループ」に注意が必要。rubyの場合 ctrlキー + c キー で強制終了

使う繰り返し構文について

whileとtimes等、どちらを使うか迷ったときは繰り返す回数が分かっているかで考える。

```
10回繰り返す    ... times  
〇〇以下の間等  ... while
```

後判定のwhile文

通常のwhile文は条件を満たさないときは、ループの中は一度も実行されない。

```
a = 10  
while a < 10  
  puts "こんにちは"  
end
```

ループの中を一度は実行したいときは、後判定のwhileを使う

```
a = 10  
  
begin  
  puts "こんにちは"  
end while a < 10
```

複合代入演算子

和差積商などの演算子と代入演算子=を組み合わせたもの

- `a += b` ... aにbを足したものをaに入れる。 `a = a + b` と同じ
- `a -= b` ... aからbを引いたものをaに入れる。 `a = a - b` と同じ

- `a *= b` ... `a`と`b`を掛けたものを`a`に入れる。 `a = a * b`と同じ
- `a /= b` ... `a`を`b`で割ったものを`a`に入れる。 `a = a / b`と同じ
- `a %= b` ... `a`を`b`で割ったときの余りを`a`に入れる。 `a = a % b`と同じ

開始値から終了値までのループ

```
開始値.upto(終了値) do |変数|  
  処理  
end
```

※変数には開始値から終了値までが、1ずつ増やえながら入る

練習問題

キーボードから値を入力し、1からその値までの数と、その数分の*を出力

```
値を入力：5  
1:*  
2:**  
3:***  
4:****  
5:*****
```

配列

一つの変数の中に複数の値を入れるもの。

rubyの変数には型がないため、同じ配列の中に文字列型や数値型を混合させて入れることができる。

配列の定義

```
配列名 = []
```

宣言と同時に値を入れるには

```
配列名 = [値1, 値2, 値3, ...]
```

とする

要素とインデックス

配列に入っている値のことを要素という

配列に値を入れると先頭の要素から、0から始まる番号が振られる。この番号のことをインデックスという

※配列名 = ["a","b","c"]...aのインデックスは0、bは1、cは2になる

配列に値を代入する

```
配列名[インデックス] = 値
```

配列の値を参照する

```
配列名[インデックス]
```

p メソッド

pメソッドはその後に指定された変数などの中身を表示するものになる

表示する値の型によって出力が変わる

```
数値型  
そのまま出力  
例 1
```

```
文字列型  
前後を""で囲んで出力  
例 "abc"
```

```
配列型  
[]で囲んですべての要素を出力  
例 [1,2,3]
```

```
nil  
空（から）を表す値。  
c言語ではnull
```

配列のループ

配列のループには通常eachメソッドを使う

```
配列名.each do |変数|
  処理
end
```

変数には、先頭から要素が代入され、全ての要素を取り出したらループが終わる

文字列型

文字列を表すための型。

文字列にする部分を、"（ダブルクォート）または'（シングルクォート）で囲む必要がある。

例 "ABC" 'ABC'

エスケープシーケンス

\（バックスラッシュ）と英字で表される特殊な文字 Windowsの場合、¥になるときもある。 主な
エスケープシーケンス \t ... 水平タブ /" ... "\n ... 改行 /' ... '\ ... \

式展開 文字列の中に変数や計算式を埋め込むもの 文字列の中に#{変数}で表す

例 tanka = 100 price = 3 puts "単価#{tanka}円のを#{price}個買うと#{tanka * price}円"

単価100円のを3個買うと300円と表示される

"と'の違い 文字列は"か'で囲む必要があるが、どちらで囲むかによって動作に違いがある。 ・エスケープシーケンスが一部しか使えない。（\と'だけ使える） ・式展開が出来ない

ヒアドキュメント 改行を含んだ文字列を設定するもの。 入力されている文字列がそのまま文字列になる。 <<終了文字列の次の行から行頭にある終了文字列で囲む

<<EOS a ab abc EOS

ヒアドキュメントのなかではシングルクォートやダブルクォートはそのまま出力されるが、 #{の式展開は実行される。 は行頭に書く必要があり、その前にはスペースも入れれない。 終了文字の<<と終了文字の間に-を入れると行頭に書く必要は無くなる。

シンボルクラス 様々なものの名前を表すクラス。 プログラムのコード上では文字列、中では数値で動作する :シンボル名で表す 主にハッシュや引数などで利用される。

title = :name puts title -> nameと出力される

論理クラス（Booleanクラス） 正しい（真、true）か正しくない（偽、false）を表すクラス nilクラス 空（から）をあらわすクラス。0や""（長さ0の文字列）とは異なるので注意

trueやfalse、nilのことを擬似変数という。

if文等の真偽判定 rubyの場合、falseとnil以外は全てtrueになる

`a ||= 0`の意味 `a ||= 0` は `a = a || 0` と同じ意味になる `a`がnilかfalseの時には0を入れるという意味になり、`a`が初期化されていないときは0を入れ、それ以外のときは`a`の値をそのまま使うという意味になる。 rubyの場合、よく使う書き方になる

配列クラス 一つの変数の中に複数の値が入っているものを配列という 配列に入っているそれぞれの値を要素といい、要素には0から始まる番号が自動的に振られる。 この番号のことをインデックスという

配列の宣言 主に3種類の宣言方法がある

[]を使う方法 配列名 = [] []を使い、配列の宣言と同時に値を入れる 配列名 = [値1,値2,値3,値4,...]

Array.newを使う 配列名 = Array.new ※配列名 = []と同じ意味

配列を使う理由 配列は同じ種類や内容のデータをまとめて扱うことに適している。

例、 ・データの件数の取得 ・次のデータのアドレス管理 ・引数にして受け渡し

等々

要素に値を設定する 配列名[インデックス] = 値

要素の値を参照する 配列名[インデックス]

配列に要素を追加する 配列名 << 値

rubyの配列について 現在の最大インデックスより、大きいインデックスを指定し 値を設定すると、指定したインデックスの要素に値が入り、 その間の要素にはnilが入る またインデックスにマイナスの値を設定すると、 末尾からインデックスが指定できる

%wや%iを使った配列の作成 文字列型やシンボル型の配列を作成するとき %を使った指定の方法がある

文字列型の配列 %w|a b c| => ["a","b","c"] ※スペース区切りで文字列型の配列になる シン

ボル型の配列 %i|a b c| => [:a,:b,:c] ※スペース区切りでシンボル型の配列になる

|の他に!や()や{}など、英数字以外だったら何で困っても良い また式展開やエスケープシーケンスを使うときには、それぞれ大文字 にする。シンボルを使ったハッシュ ハッシュのキーは、多くの場合シンボルが利用される hash = {シンボル名 => 値,シンボル名 => 値,...}

シンボルを使ったハッシュの場合特別な宣言がある

hash = {シンボル名: 値,シンボル名: 値,...}

上の宣言と下の宣言は同じものになる

ハッシュのeach

ハッシュに対してもeach文が使える 配列のハッシュと異なるのはキー用の変数と値用の変数が必要になること

ハッシュ.each do |キー用の変数,値用の変数| 処理 end

変数を一つしか指定しない場合、変数には[キー,値]の配列で 値が入る。

ユーザ定義メソッド

今まで使用していたputsやprintはシステムが定義しているメソッドになる。このメソッドをユーザが作成するものをユーザ定義メソッドという。

ユーザ定義メソッドの作り方

```
def メソッド名(引数,引数,...)  処理  end    ※引数は省略可
```

ユーザ定義メソッドの呼び出し

メソッド名(引数,引数,...)

ユーザ定義メソッドの命名

アルファベットの小文字ではじめる。2文字目からはアルファベット、数字、アンダーバーを使う。メソッド名を見たら何をするメソッドか分かるようにする。

戻り値について

メソッドから呼び出しもとに何らかの値を1つだけ返すことができる。この値のことを戻り値という。メソッド内で `return 値` とすると、指定した値が戻り値になる。またrubyの場合、最後に実行したメソッドの値が自動で戻り値になる。

変数のスコープ

変数ができる範囲を変数のスコープ（有効範囲）という。英小文字または_から始まる ... ローカル変数、\$から始まる ... グローバル変数（プログラムの中で共通）、@から始まる ... インスタンス変数、@@から始まる ... クラス変数、英大文字から始まる ... 定数(一度設定したら書き換えれない変数) 但しRubyの場合、警告が出るが変更できる。

オブジェクト指向プログラミングとは

処理対象を一つのモノ（オブジェクト）として扱い、その集合体で一つのシステムを作るという考え方。

オブジェクトにはオブジェクトが持つ値（フィールド、プロパティ）とオブジェクトが行う動作（メソッド）がある。例、車というシステム。車は、タイヤ、エンジン、ハンドル等のオブジェクトの集合体である。そのオブジェクトがどのようなプロパティやメソッドを持っているかを定義しているのがクラスになる。クラスはオブジェクトの設計図になる。今現在、ほとんどのプログラムはオブジェクト指向で作成されている。

クラスとインスタンス

インスタンスはオブジェクトと同じ意味になる。明示的にクラスを宣言し、そのクラスからできたものをインスタンス。IntegerやStringなど、システムが作成したクラスからできたものをオブジェクトといふことが多い。クラスはあくまでもインスタンスの設計図となり、実際にデータや動作をすることは少なく、実際にデータを持ったり動作するのはインスタンスになる。

クラスの定義

クラスは以下のとおり定義を行う

```
class クラス名 def メソッド名(引数,引数,...) 処理 end def メソッド名(引数,引数,...) 処理 end end
```

クラス名は英大文字で始める必要がある。プロパティは@変数名になり、全てのメソッドで 共通で使用できる。

クラスの作成例 縦と横の長さを入力し、長方形の面積を求めるクラスShikaku class Shikaku
def tate(t) @tate = t end def yoko(y) @yoko = y end def menseki @tate * @yoko end end

インスタンスの作成

インスタンスは、クラス名.newで作成される 例えば変数にインスタンスを代入するときは 変数 = クラス名.new となる。

作成したインスタンスのメソッドにアクセスするには インスタンス名.メソッド名(引数) とする。

クラス外部からプロパティに直接アクセスする方法は無く、メソッドを 使いプロパティにアクセスする

問、 円の面積と円周を求めるクラスを作成 インスタンスを作成し以下の通り出力 但し、半径はキーボードから取得し円周率は3.14、計算式は 面積：半径×半径×円周率 円周：2×半径×円周率 とする

半径：10 面積：314 円周：62.8

問、キーボードから3科目の点数を取得 合計点と平均点を求めるクラスを作成し、以下のように出力 国語：40 数学：50 英語：61 合計：151 平均：50.33333333

セッターとゲッター

インスタンスのインスタンス変数にデータをセットするメソッドを セッターまたはライターという インスタンスのインスタンス変数からデータを取得するメソッドを ゲッターまたはリーダーという。

=を使ったセッター インスタンス変数にデータをセットするメソッドは、よく下記のように 定義される

```
def インスタンス名=(引数) インスタンス変数 = 引数 end
```

こういう風に宣言することで、クラス外からデータをセットする際 下記のように使うことができる

インスタンス名.インスタンス変数名 = 値

値に入ったデータはメソッドの引数となり、インスタンス変数に 設定したデータがセットされる。イコールの前後の空白は無視される。

インスタンス変数名=のセッターメソッドやインスタンス変数と 同名のゲッターはセッターやゲッターの基本的な命名方法になる

復習

問、キーボードから単価と数量を取得 金額を求めるクラスを作成し、以下のように出力

例 単価：100 数量：10 100円のを10個買うと1000円

問、キーボードから何月を入力 季節を判定するクラスを作成し、以下のように出力 春（3～5）、夏（6～8）、秋（9～11）、冬（12～2）

例 月：10 10月は秋です。

問、 キーボードから走行距離と走行時間を入力すると平均時速を求める。 但し、実行には以下のコードを使うこと

```
s = Sokudo.new
print "走行距離："
s.kyori = gets.to_i
print "走行時間："
s.jikan = gets.to_i
puts "#{s.kyori}kmを#{s.jikan}時間で走ると平均時速は#{s.jisoku}です"
```

アクセスメソッド

インスタンス変数にデータを取得したり、データを設定するメソッドを自動で定義してくれるメソッドがある。

インスタンス変数にアクセスするメソッドのことをアクセサという。

データを設定するメソッド

attr_reader :インスタンス変数名 ... ゲッターの定義
attr_writer :インスタンス変数名 ... セッターの定義
attr_accessor :インスタンス変数名 ...セッターとゲッターの両方を宣言
※インスタンス変数は,で区切ることができる。

復習

問

名前と時間を設定すると挨拶を行うクラス
3～10：おはようございます 11～18:こんにちは 19～2:こんばんわ

```
aisatsu = Aisatsu.new
print "名前："
aisatsu.name = gets.chomp
print "時間："
aisatsu.jikan = gets.to_i
puts "#{aisatsu.aisatsu}。#{aisatsu.name}さん"
```

問


```
以下のクラスを作成
クラス名 : Jikyu
プロパティ : jikyu ... 時給
hour ... 時間
name ... 名前
メソッド : kyuyo ... 給与
```

〇〇さんは時給〇円で〇時間働き〇円

```
問 以下のクラスを作成      クラス名 : Jikyu      プロパティ : jikyu ... 時給
hour ... 時間                yakin ... 夜勤の時間        name ... 名前      メソッド :
kyuyo ... 給与              但し夜勤の時給は+100円とする
```

出力例 時給は1000円、時間は10時間、夜勤は5時間の坂根さんの時給

坂根さんは15500円

initializeメソッド

インスタンスを作成時に処理を行うメソッド。

```
class Sample def initialize 処理      end end s = Sample.new <- ここでinitializeメソッドが実行される
```

主にインスタンス変数の初期化等に利用される。 このようにインスタンスが作成されるときに、自動で実行されるメソッド を、コンストラクタという。

initializeメソッドに引数を渡すこともできる

```
class Sample
  def initialize(引数1,引数2,...)
    処理
  end
end
s = Sample.new(引数1,引数2,...)
```

self変数について

selfは自分自身を表す変数になる。 基本的には使う機会は少ないが、ローカル変数と同じ名前のメソッドがある場合など、メソッドを呼び出すときに使用される。

```
class Sample
  def method
    10
  end
  def sample(method)
    print method
  end
end
```

```
end
```

```
a = Sample.new  
a.sample(100)
```

P94

配列を使った値の代入

複数の変数に配列を使って値を代入することができる

```
a,b,c = [1,2,3]  
または  
arr = [1,2,3]  
a,b,c = arr
```

両方とも、aには1、bには2、cには3が代入される

配列の要素が足りない場合

値を順番に入れていって、足りないところにはnilが入る

```
arr = [1,2]  
a,b,c = arr ... aは1、bは2、cはnil
```

配列の要素が多い場合

値を順番に入れていって、過分は無視される

```
arr = [1,2,3,4]  
a,b,c = arr
```

*を使った代入

1つだけ*をつけて変数を指定できる。
その場合、ほかの変数には順番に値を入れて、残りがすべて
*を付けた変数に配列として入る。

```
arr = [1,2,3,4]  
a, *b, c = arr #aは1、bは[2,3]、cは4
```

```
arr = [1,2,3]
a, *b, c = arr  #aは1,bは[2],cは3

arr = [1,3]
a, *b, c = arr  #aは1,bは[],cは3
```

UFO比較演算子

主に、ソートなどに使われる演算子。

左辺と右辺を比べて、
左辺が大きければ - 1
同じだったら 0
右辺が大きければ 1
が返される

式1 <=> 式2

同一性と同値性

同じ値か同じオブジェクト化のこと
同一性・・同じオブジェクトか？
同値性・・同じ値か？

```
a = "1"
b = "1"
```

```
puts "同じ値です if a == b"          <- 同値性のチェック 今回は真
puts "同じオブジェクトです" if a.equal?(b) <- 同一性のチェック 今回は偽
```

※a.eql?(b)は同値性のチェックのメソッドである

条件演算子

if else文を一行でかく演算子。
Railsや組み込み系でよく使われる

条件 ? 条件が正しい時の処理 : 条件が正しくないときの処理

```
# 偶数か奇数かを求めるプログラム
1.upto(10) do |i|
```

```
puts "#{i}は#{i % 2 == 0 ? '偶数' : '奇数'}です"  
end
```

組み込みを使うのに適したプログラム

```
a.equal?(b)  
"#{i}は#{i % 2 == 0 ? "偶数" : "奇数"}です"
```

のような複数行を一つの行にまとめる技術はRailsなどで重宝される

論理演算子を使ったプログラムの実行

```
a ||= b ...  
a &&= b ...
```

`a ||= b`は`a = a || b`のことである。ここで`a`が`false`か`nil`だった場合、`b`が次に判定される仕組みになっているので、`a`に`false`か`nil`以外の値が入っていたら`b`は判定にかけられない

```
a = nil  
b = 3  
  
a ||= b  
  
puts a
```

Bash

```
3
```

`||=` と `&&=` の活用

`a ||=` メソッドは
(`a`もしくはメソッド)の返回值ではなく、
`a`に`nil`か`false`が入っていたらメソッドを実行する

逆に

`a &&= a.to_i`は

`a = a && a.to_i`で`a`が未定義ではなかった場合、`a.to_i`を実行する。この式では`a`が`nil`だった場合判定にかけない仕組みになっており、`false`は確定なので`a.to_i`を実行しない。

`a ||= b ...` `a`が未定義だったら、`a`に`b`を入れる

`a &&= b` ... `a`と`b`が未定義されていたら、`a`に`b`を入れる。

`a && メソッド` ... `a`が`nil`か`False`以外であれば、メソッドを実行する

また、`&.` という演算子がある

`a & . to_i` ... `a`が`nil`以外だったら、`a.to_i`を実行する

この演算子をぼっち演算子

ビット演算子

upcaseメソッドとは

大文字にするメソッドが`upcase`

```
str = "sakane"  
p str&.upcase
```

Bash

```
SAKANE
```

読み込んだら絶対メモリを確保する演算子

```
if false  
  str = "sakane"  
end  
p str  
p str&.upcase
```

ここで`false`の場所は絶対`false`だから実行されないかと思われるが、実はプログラムを全部読み込むときに一応、`str`のメモリを確保してあるから`str`に格納される

```
if true  
  str = 0  
end  
p str  
p str&.upcase
```

ここでupcaseを実行したときInteger型にはupcaseはないですよ。ということでエラーになる。明確な理由としては文字は1バイトであるため、Integer型から大文字を作る理由がないため

ビット演算子

2新数に対して論理和、論理積、排他的論理和を行うメソッド

```
& ... 論理積
| ... 論理和
^ ... 排他的論理和
~ ... 否定
<< ... 左シフト
>> ... 右シフト
```

2新数のリテラル値

0bからは始まる値は二進数として処理される 0oからは始まる値は8進数として処理される 0dからは始まる値は10進数として処理される 0xからは始まる値は16進数として処理される

2進数での出力

.to_s(2)

演算子の優先順位

演算子には優先順位がある。考慮しておかないと思わぬエラーを起こすので注意する。またsilver試験でよく出てくる問題にもなる。

```
::          クラスなどの定数の利用
[]          配列
\+ , ! , ~  プラスの数や定数を表す
**          べき乗
*,/,%       演算
+,-         演算
<< , >>     シフト演算
&           ビット演算
| , ^       ビット演算
> , >= , <= , < 比較演算
==,===,<=> , !=, =~, !~
比較演算子、正規表現の比較演算子
&&         論理演算子
||         論理演算子
.. , ...    範囲演算子
?:         条件演算子
=          代入演算子
not        論理演算子
and,or     論理演算子
```

結合則

同じ順位の比較演算子が並んでいるとき、右側から計算するか左から計算するかの考え方

左結合

```
a = 1 + 2 + 3
```

右結合

```
a = b = 1  
a == b == c
```

無限ループ

無限ループを行いたいときは、

```
loop do  
  処理  
end  
とする
```

ただし、プログラムを停止しないとプログラムが終了しないので注意が必要

```
while true  
  処理  
end
```

という書き方がある

ループ制御

break 処理を中断しループを抜ける next 次のループにスキップする redo 同じループをはじめてから行う

多重ループから一気に抜ける

多重ループで処理を行う場合、breakなどの対象になるのは、直近のループだけになる。

全てのループから抜きたい場合、catchとthrowを使う。

```
catch ブロック名 do  
  処理  
  throw ブロック名  
end
```

※Javaのcatch,throwがあるが、まったく別の動きをするので注意が必要

例外処理

プログラム実行中に起こるエラーのことを例外という。

例外が発生するとプログラムが停止してしまうが、停止しないようにすることを例外処理という。

```
begin
  例外が発生しそうな処理
rescue
  例外が発生したときの処理
else
  例外が発生しなかったときの処理
ensure
  例外が発生してもしなくても実行する処理
end
```

※elseやensureは省略可能

例外の種類によって処理を変える

例外もクラスから作成されたオブジェクトになる
rescueの後ろにクラス名を指定することで取得する例外を指定することができる

```
begin
  例外が発生しそうな処理
rescue -> クラス名(エラーに関するクラス)
  指定したクラスの例外処理
rescue
  それ以外の例外処理
end
```

例外を変数に入れる

例外もオブジェクトの一つになるので変数に入れることができる

```
rescue[例外クラス] => 変数
```

例外クラスには以下のメソッドがある。


```
.class      例外の種類を返す  
.backtrace  例外の発生場所を返す  
.message    例外のメッセージを返す
```

ここまででの補足

テキストファイルを開いた状態でプログラム終了した場合、そのテキストファイルのロックがかかったままになるのでそのファイルは開けなくなる。

例外発生時にリトライする

例外が発生したときにbeginまで戻って再実行することもできる。例外処理の中にretryと記述する。

```
rescue  
  retry
```

他のファイルを参照する P181

他のクラスファイルやライブラリファイルを自分自身に組み込んで使用することがある。

```
require "ライブラリ名"
```

クラスの定義を別ファイルにしたり、Mix-inと呼ばれる機能を使うときに使用される

※ライブラリ ... 共通で使うメソッドを登録してあるファイルシステムが準備している標準ライブラリや自分で作成したユーザ定義ライブラリなどがある

Mix-in ... 自作クラスにライブラリファイルを読み込んで使うもの

文字列クラスのメソッド P184

文字数を取得する

```
文字列.length  
文字列.size
```

バイト数を取得する

```
文字数.bytesize
```

委譲の考え方

クラスが持つべき機能を、他のクラスで作成し、そのクラスのオブジェクトにインスタンス変数に持つことで、クラスを完成させること。

Main.rb

```
t = Taiseki.new(En.new(10),20)
```

Taiseki.rb

```
def initialize(zukei,takasa) <- zukeiは持ってきたクラスのアドレス
@zukei,@takasa = zukei,takasa
end

@zukei.r <-持ってきたアドレスの参照先からプロパティを参照できる
```

継承

基となるクラスを拡張し、新しいクラスを作ること。

基となるクラスをスーパークラス、新しく作成されるクラスをサブクラスという。

```
class サブクラス名 < スーパークラス
  拡張するプロパティやメソッド
end
```

サブクラスは、スーパークラスの全てのインスタンス変数やメソッドを使用することができる。

継承の種類

サブクラスが複数のスーパーを持つことを多重継承。一つのスーパークラスしか持てない継承を単一継承という。

※rubyは単一継承しかできない言語になる。

単一継承の言語...Java、Ruby、C#
多重継承の言語...C++、Python

継承の考え方

is-aの関係で考える

車クラスを継承して、スーパーカークラスを作成する

スーパーカー is a 車

車クラスに走行の仕組みを作成し、スーパーカーにしかない機能を継承したスーパーカークラスに作成する

```
class Sankaku < Shikakukei #->これはダメ。なぜならis-a関係になっていないから
  def menseki
    super / 2
  end
end
```

オーバーライド

スーパークラスのメソッドをサブクラスで上書きすること

継承クラス

必ず継承されて、必ずオーバーライドされるメソッドを持つクラス。

継承クラスからインスタンス作成することはできない

抽象クラス

クラスは継承するが、全メソッドはオーバーライドされるメソッドのこと

クラスメソッド

インスタンスではなく、クラスにメソッドを持たせることができる

定義法

```
class クラス名
  def self.メソッド
    クラスメソッドの処理
  end
end
```

実行

クラス名.メソッド名

例：

```
class Chouhoukei
  def self.makeSeihoukei(hen) #ここがクラスメソッド
    Chouhoukei.new(hen,hen) #ここでインスタンス定義しているが、返り値がインスタンス定義になるの
    で、アドレスを返すことになる。
  end
  def initialize(tate,yoko) #インスタンスメソッド?
    @tate,@yoko = tate,yoko
  end
  def menseki #ここがインスタンスメソッド (インスタンス化できるからインスタンスメソッド)
    @tate * @yoko
  end
end
```

クラス変数

@@から始まる。そのクラスから作成されたインスタンスで共通で使用できる。クラスの先頭で初期化を行うことが多い。

定義法

```
class クラス名
  @@変数名 = 値
  def メソッド
    クラス変数を使った処理
  end
end
```

例

```
class Car
  @@count = 0
  def initialize
    puts "車を作成しました"
    @@count += 1
  end
  def self.count #クラスメソッドを使うべきであるから -> インスタンスメソッドはインスタンス変数(@変数名)があるから、インスタンスメソッドでそのインスタンス独自の処理をする
    puts "車を#{@count}台作りました"
  end
end
```

クラス定数

クラス内に定義された定数。クラス内部からは定数名だけ、クラス外部からはクラス名::定数名でアクセスできる

定義

```
class クラス名
  定数名 = 値
  def メソッド
    定数名
  end
end
```

アクセス
クラス名::定数名

例

```
class En
  PI = 3.141592
  def initialize(r)
    @r = r
  end
  def menseki
    @r * @r * PI
  end
end
```

定数はrubyの場合、変更できる

```
En::PI = 3.14 #警告が出力される
puts En::PI
```

完全定数を作る場合

```
class En
  PI = 3.141592.freeze
end
```

::の使用用途

::でメソッドを呼び出せるが、基本定数のみを参照するのが普通

定数のオーバーライドの注意点

```
class Uriage #計算して金額を出すクラス
  Tax = 0.1 #税率が10% (物の税率は10%)

  attr_reader :tanka, :cnt, :kingaku, :name
  def initialize(name,tanka,cnt)
    @name,@tanka,@cnt = name,tanka,cnt
  end
  def tax
    (Tax * kingaku).to_i
  end
  def kingaku
    @tanka * @cnt
  end
  def total
    kingaku + tax
  end
end
```

```
class Uriage_food < Uriage #売上のクラスを継承
  Tax = 0.08 #食品の税率は8% (軽減税率という)

  def tax #この場合、taxをもう一度定義しないと、親クラスのtaxを使ってしまうことになる
    (Tax * kingaku).to_i
  end
end
```

カプセル化について

オブジェクト指向の考え方の一つにカプセル化がある。

クラス外部に公開するのは最小限にし、インスタンス変数には、必ずメソッドを介してアクセスするというものになる。

カプセル化のメリット

- 必要最小限の情報しか公開しないことで、悪意のあるユーザーからの攻撃に強くなる。
- 必ずメソッドを介してインスタンス変数にアクセスすることで、不正なデータがセットされることを防ぐ

例えば、[データベース] <- フルネームを返すクラス..をするとどんな値が返ってくるかで、データベース構造がわかってしまう。それでどんな方法でも同じ値を返すような、データベース構造をわからなくするような設計をカプセル化となる。

アクセス修飾子

そのメソッドが使える範囲を指定するもの

- public ... インスタンス内部と外部どちらからも使用可能
- private ... インスタンス内部だけで使用可能
- protected ... 同一クラスで作成されたインスタンス内部で使用可能

※特に指定しない場合、publicになる

アクセス修飾子の使い方

```
class クラス名
  public
  def パブリックメソッド名
    処理
  end
  def パブリックメソッド名
    処理
  end

  private
  def プライベートメソッド名
    処理
  end

  protected
  def プロテクトドメソッド名
    処理
  end
end
```

protectedの使い方

```
class Name
  def initialize(name)
    @name = name
  end
  def return_name(obj = self)
    obj.name
  end
  def return_protected_name(obj = self) #ここでインスタンス変数を定義している
    obj.name #これがクラス内部のインスタンス
  end

  protected
  def name
    @name
  end
end
```

```
end

n = Name.new("坂根")
puts "return_name"
puts n.return_protected_name
```

後からでもアクセスの範囲を設定できる。

```
private :メソッド名, :メソッド名...
protected :メソッド名, :メソッド名...
```

指定するメソッドは定義済みの必要がある。

ポリモーフィズム

ポリモルフィズムや多様性とも言われる。

同じような仕事をするメソッドであれば、同じ名前にする考え方

例、三角形の面積を求めるクラスSankakuと
四角形の面積を求めるクラスShikakuを作成する

面積の求め方は異なるが、どちらも縦と横のプロパティを持ち、面積を返すメソッドがある。

ポリモーフィズムを考えるときに継承や抽象クラスの考え方が必要になる。

ダックタイピング

「アヒルのように歩き、アヒルのように鳴くのであれば、アヒルに違いない」という言葉からできた。

オブジェクトの種類を決めるのは、名前ではなくその動作という考え方。継承などを使わずにポリモーフィズムを行うときに使われる。

```
zukei = [Sankaku.new(10,20),shi = Shikaku.new(10,20)]

zukei.each do |z|
  puts z.menseki
end
```

raise

raiseは例外処理を意図的に発生させることのできるコード


```
raise ZeroDivisionError, "ゼロで割りました"  
# ./Main.rb:17:in `<main>': ゼロで割りました (ZeroDivisionError)が出力される
```

書き方
raise エラークラス , 自由にエラーメッセージ

オリジナルの例外を作成する

クラスの内部よりオリジナルの例外を発生させることがある。オリジナルの例外を発生させるには、StandardErrorを継承したクラスを作成し、その例外をraiseを使い発生させる。

エラーメッセージを作る方法の2パターン

- raiseを使う

```
raise RangeError, "範囲外が検出されました" if point < 0 || point > 100
```

- クラスのmessageメソッドをオーバーライドする

```
class RangeError < StandardError  
  def message  
    "範囲外が検出されました"  
  end  
end
```

オープンクラス

Rubyの場合、システムが定義したクラスもメソッドを追加したり、オーバーライドや継承ができる。このような仕組みをオープンクラスという。

```
class Integer  
  def +(atai)  
    self * atai  
  end  
end  
  
puts 10 + 10
```

変わったメソッドの考え方

```
class Integer
  def self.aisatsu
    "integer型"
  end
  def +(atai)
    self * atai
  end
end

#これは10というメソッドの+メソッドを呼び出して、20は引数
puts 10 + 20
```

特異メソッドと特異クラス

インスタンスに対し、メソッドを追加やオーバーライドすることができる。追加されたメソッドを特異メソッド、追加されたインスタンスを特異クラスという。

```
class Shikaku
  def initialize(tate,yoko)
    @tate, @yoko = tate, yoko
  end
  def menseki
    @tate * @yoko
  end
end

shikaku = Shikaku.new(10,20)
sankaku = Shikaku.new(10,20) #<-特異クラス

def sankaku.menseki
  @tate * @yoko / 2
end

puts shikaku.menseki
puts sankaku.menseki
```

構造体 (Struct)

オブジェクトはデータとメソッドの集まりになる。データだけのオブジェクトを作る必要がある時は、構造体を定義する。

構造体名 = Struct.new(:フィールド名, :フィールド名, ...)

※データの塊が構造体、メソッドのあるデータの塊がオブジェクト

構造体にメソッドを追加する

構造体は通常データの集まりで、メソッドは存在しないが、rubyの場合メソッドを書くこともできる

```
構造体名 = Struct.new(:フィールド名, :フィールド名, ...) do
  def メソッド名
    処理
  end
end
```

モジュール

構造体がデータの集まりであるなら、モジュールはメソッド定数の集まりになる。

様々なクラスやプログラムで共通で使えるメソッドを定義しておき、それを呼び出すことでプログラムの再利用性を高める

モジュールとクラスの違い

- モジュールはインスタンスを作成できない

あくまでメソッドや定数は呼び出すことで利用される

- 継承できない

継承してオーバーライドするような使い方はしない。追加したい項目があれば該当モジュールに普通に追加する

モジュールの定義

```
module モジュール名
  def メソッド名
    処理
  end
  module_function :外部から呼び出すことができるメソッド名, ...
end
```

モジュールで定義されたメソッドはモジュール内からしか呼び出すことができず、モジュール外部から呼び出すときは`module_function`でメソッドを指定する。モジュール名は英大文字から開始する。

モジュールで定義したメソッドの呼び出し

主に二つの呼び出し方法がある。

- モジュール名を付けて呼び出す。

モジュール名.メソッド名で呼び出す。

- includeで読み込んで使用する

include モジュール名 とすることでメソッド名だけで呼び出されるようになる また、module_functionで指定していないメソッドも呼び出すことができる

Mix-in（ミックスイン）

クラスの中にモジュールを詰め込むことをミックスインという

```
# モジュールの作成
module Keisan
  def shishagonyu(atai)
    ((atai * 10) + 0.5).to_i / 10.0
  end
  module_function :shishagonyu
end

# 面積クラスの作成
class Menseki
  def menseki
  end
end

# 面積クラスを継承し、四角クラスを作成
class Shikaku < Menseki
  def
    initialize(tate,yoko)
    @tate,@yoko = tate,yoko
  end
  def menseki
    @tate * @yoko
  end
end

# モジュールを取り込み、面積クラスを継承した、円クラスを作成
class En < Menseki
  include Keisan
  def initialize(r)
    @r = r
  end
  def menseki
    shishagonyu(@r * @r * Math::PI)
  end
end
```

モジュールを使った特異クラス

モジュールを使うことで特異クラスを作成することができる

オブジェクト.extend モジュール名

名前空間

同名のメソッドや変数がある場合、その名前の前に識別できるものを追加する

Rubyの場合、モジュールを使うことで、モジュール名がそのまま名前空間にすることができる

クラスを名前空間を使って定義する方法

```
module モジュール名
  class クラス名
    処理
  end
end

インスタンス名 = モジュール名::クラス名.new
```

応用

```
class モジュール名::クラス名
  処理
end

# 呼び出し時
モジュール名::クラス名
```

正規表現とは

文字列のパターンチェックなどで使用されるもの

正規表現の記述方法

/パターン/

パターンと一致したかのチェック

制企業減を使用しパターンチェックを行いあっていることを、マッチするという

主なチェックは以下の通り

```
if 正規表現 =~ 文字列
  マッチしたときの処理
else
  アンマッチの時の処理
end
```

パターンの書き方

- `/XYZ/ ... XYZ`と一致するか
- `/[XYZ]/ ... []`の中の一文字が含まれるかどうか
- `/[^XYZ]/ ... []`野中以外の一文字が含まれるかどうか
- `/[X-Z]/ ... XからZまでの一文字が含まれるか`
- `/[X|Y|Z]/ ... XかYかZと一致するか`
- `/[X*]/ ... Xから始まる任意の1文字以上の文字列`
- `*` ... 直前の文字を0文字以上繰り返す。しかし直前の文字があれば1回以上繰り返す
- `+` ... 直前の文字を1文字以上繰り返す
- `?` ... 直前の文字を0回か1回含む
- `{数字}` ... 直前の文字を数字の回数繰り返す
- `{最小,最大}` ... 直前の文字を最小~最大の回数繰り返す（どちらか片方は省略できる。
例：`/ab{4}c/ .. 4文字以下`

```
# abcが含まれているかどうか
str = "xyzabc"

if /abc/ =~ str ... True
```

```
# aかbかcかいずれかが含まれてるか
str = "axbycz"

if /[abc]/ =~ str ... True
if /abc/ =~ str ... False
```

```
# a,b,c以外の文字が含まれているか
str = "abc"
str2 = "abcz"

if /^[abc]/ =~ str ... False
if /^[abc]/ =~ str2 ... True
```

```
# 1 ~ 9までの任意の一文字が含まれている
str = "abc3"
str2 = "11"

if /[1-9]/ =~ str ... True
if /[1-9]/ =~ str2 ... True
```

```
# 直前の0文字以上を繰り返す
str = "abc"
str2 = "abdc"
str3 = "ac"

if /ab*c/ =~ str ... True
if /ab*c/ =~ str2 ... False
if /ab*c/ =~ str3 ... True
```

```
# 直前の文字を1文字以上繰り返す
str = "abc"
str2 = "ac"

if /ab+c/ =~ str ... True
if /ab+c/ =~ str2 ... False
```

```
# 直前の文字を0回か1回含む
str1 = "abc"
str2 = "ac"
str3 = "abbc"

if /ab?c/ =~ str1 ... True
if /ab?c/ =~ str2 ... True
if /ab?c/ =~ str3 ... False
```

```
# 直前の文字を指定回数分繰り返す
str = "abbbbc"
str2 = "abbbbbc"

if /ab{4}c/ =~ str ... True
if /ab{3,6}c/ =~ str ... True
if /ab{,4}c/ =~ str2 ... False
```

位置の指定

- `^` ... 正規表現開始の/`/`のすぐ後に書くことによって行の先頭を表す

```
# abc abcd abcc = True
# cabcd dab aabc = False
if /^abc/ =~ str
```

- `$` ... 正規表現終了の/`/`の直前に書くことで行末を表す。

```
# dabc eeabc dfabc = True
# abcd aacb bc = False
if /abc$/ =~ str
```

文字を使った正規表現

- `.` (ドット) ... 任意の一文字 (数字でも記号でも英語でも可)
- `\w` ... 英字、数字、アンダースコアと一致
- `\W` ... 文字以外と一致。`[\^\w]`と同じ
- `\d` ... 0から9に一致
- `\D` ... 数値以外と一致
- `\n` ... 改行と一致
- `\t` ... タブ文字と一致
- `\s` ... 空白と一致
- `\S` ... 空白以外と一致

文字を使った位置の指定

- `/a` ... 正規表現開始の/`/`のすぐ後ろに書くことによって行の先頭を表す
- `z/` ... 正規表現終了の/`/`の直前に書くことで行末を表す
- `Z/` ... 正規表現終了の/`/`の直前に書くことで行末を表す (行末が改行だったら改行の前)

正規表現を変数に入れる

正規表現も一つのインスタンスとして扱うことができる。

以下の書き方で正規表現を変数に入れることができる


```
変数 = Regexp.new("パターン") #ここではシングルクォートで囲まないと、エスケープシーケンスが効かなくなる
変数 = /パターン/
変数 = %r{パターン} #※{}は()や[], !, ||などに変更可
```

パターンマッチについて

正規表現を用いて、パターンマッチしたかのチェックには主に下記が使用される

```
パターン =~ 文字列 ... 最初にパターンマッチした場所を返す
パターン === 文字列 ... 正規表現にマッチしたかを返す (true/false)
パターン.match(文字列) ... パターンマッチ前、パターンマッチ部分、パターンマッチ後等を返す
```

matchメソッドについて

matchメソッドで取得したデータは以下のデータが入っている

```
[0] ... マッチした部分が入っている
[n] ... n番目の{}にマッチした部分を返す
pre_match ... マッチした部分より前
post_match ... マッチした部分より後
```

matchを使う場合、ブロックを使うときもある

```
パターン.match(文字列) do |変数|
  変数を使った処理
end
```

例文

```
regex = /\d{2}-\d{4}/

print "文字列："
str = gets.chomp
p regex.match(str)
```

実行結果

```
PS C:\Users\student\rubysrc\sample31> ruby Main.rb
文字列：12345
nil
PS C:\Users\student\rubysrc\sample31> ruby Main.rb
```

```
文字列：12-2345
#<MatchData "12-2345"> ... マッチしたからこれが入っている
```

配列ごとに何が入っているかイメージする

```
regex = /(\d{2})-(\d{4})/ #()で囲った部分を単位として配列の要素にマッチを記述する

print "文字列："
str2 = gets.chomp
result = regex.match(str2)
p result[0]
p result[1]
p result[2]
p result.pre_match
p result.post_match
```

実行結果

```
PS C:\Users\student\rubysrc\sample31> ruby Main.rb
文字列：12345-45-6789-2345
"45-6789"
"45"
"6789"
"12345- "
"-2345"
PS C:\Users\student\rubysrc\sample31>
```

イメージ

```
12345- 45-6789 -2345
| pre | match | post |
```

パターンマッチする箇所が何か所がある場合、最長の部分にマッチする

```
result = /a.*z/.match("abczabczabcz") # aの次に任意の文字列が1文字以上あり、次がzである
p result[0]
```

実行結果

```
"abczabczabcz"
# abcz と abcz と abcz と abczabcz と abczabcz と abczabczabczが存在する
```

正規表現のオプション

正規表現には様々なオプションがある

- i ... 大文字小文字を区別しない
- x ... 空白やコメントを含めない
- m ... 任意の一文字に改行を含める

それぞれ正規表現の直後に指定する

```
# /パターン/オプション  
/[a-z]{4}/i
```

最短マッチ

正規表現を使ってマッチする文字列を調べると最長の区間がマッチする。

最短の部分をマッチさせるには

長さ指定?

とする。

```
# 例  
# aから始まりcで終わる最短パターン  
/a.*?c/
```

実際のコード -> 出力

sample

```
str = "abcabcabc"  
result = /(a.*c)/.match(str)  
p result[0]  
  
str = "abcabcabc"  
result = /(a.*?c)/.match(str)  
p result[0]
```

実行結果

```
"abcabcabc"  
"abc"
```

文字列の置き換え

正規表現を使って文字列を置き換えるときはgsubメソッドを使う。

```
# マッチした場所を置き換える  
文字列.gsub(パターン, 置き換える文字)
```

ただし、gsubは文字列を置き換えるのではなく、置き換えた内容を戻すメソッドになり、元の文字列は変更しない。このように実行したインスタンスの内容を変更しないメソッドを非破壊的メソッドという。

またgsub!とすると元の文字列を変更する。このようなメソッドを破壊的メソッドという。

また、パターンマッチした先頭だけ置き換えることもできる。

```
文字列.sub(パターン, 文字列)
```

subも破壊的メソッドとなり、破壊的メソッドはsub!となる

ここまでがテスト範囲
